Software Requirements & Design Description

Course Team Project - Deliverable #1

CE/CS/SE 3354 Software Engineering

0. Document Control

Project Title	Campus Safety Incident & Emergency Accountability System
Team Name / #	**SafeGuard** / 03
GitHub Repository URL	https://github.com/kouroshutd/F26_3354_03
Report Date / Version	October 25, 2026 - v1.0

1. Task Delegation

Delegation of tasks for Deliverable #1, listing **every** team member and what they contributed. Members with no or limited contribution are noted explicitly.

Team Member	Task(s) for Deliverable #1	Contribution Status
Kourosh Torkaman Sohrabi	Project management; architectural design (5.3); Sections 0, 1, 2, 3, 6; GitHub setup tasks 1.2, 1.3, 1.6; assembly and submission	Full
Quinn Merida Friman	Use case diagram corrections (4.3); traceability matrix verification (4.4)	Full
Neelesh Polireddy	Sequence diagram UC-03 Conduct Safe Walk Session (5.1); README commit, GitHub task 1.4	Full
Nihal Khan	Sequence diagram UC-01 Submit Hazard Report (5.1)	Full
Nuzhat Tabassum	Sequence diagram UC-02 Process Hazard Report (5.1)	Full
Rajas Danekar	Sequence diagram UC-04 Run Emergency Check-In Event (5.1)	Full
Quan Do	Unified class diagram (5.2); Project Scope PDF commit, GitHub task 1.5	Full

2. Feedback Addressed from Project Proposal

This section documents the feedback received on our Project Proposal / Charter and how our team responded to it in this deliverable.

Feedback Received on Proposal	How We Addressed It in Deliverable #1
No feedback was received on the Project Charter. Confirmed with the instructor on 10/12/2026 and with the TA, Lokesh Balasubramaniam, on 10/13/2026.	No changes were required. The charter's scope, objectives and feature set carry forward unchanged into this deliverable.

3. Software Process Model

Describe which software process model your team is using for this project and justify why it fits your team size, project scope, and timeline (e.g., Waterfall, Incremental, Agile/Scrum, Spiral).

3.1 Selected Process Model

Incremental development.

3.2 Justification

The system divides into five feature slices (accounts and access control, student-side reporting, the administrative queue, Safe Walk, and emergency muster) that can be specified, designed and tested independently, then integrated through three defined cross-subsystem rules. With seven members, each person carries one increment end to end rather than waiting on a shared phase, which is the main scaling problem a plan-driven model creates at this team size.

The semester runs from 8/24 to 12/09 with two graded deliverables, so the project is already structured as two increments with a fixed integration point between them. Requirements volatility is moderate: the charter scope is fixed, but grader feedback on each deliverable may change details, which an incremental model absorbs and a plan-driven model would not.

Alternatives considered: Waterfall was rejected because it assumes requirements are stable and complete before design begins. Agile/Scrum was rejected because its ceremonies assume frequent synchronous meetings that seven students with different schedules cannot sustain. Spiral was rejected because its risk-driven prototyping cycles are heavier than a one-semester project with no budget justifies.

Assumptions: (1) all seven members remain enrolled and contributing through 12/09; (2) each member owns one feature slice through requirements, design, prototype and test; (3) the three cross-subsystem rules are integrated once, after individual slices are specified; (4) at least one slice is implemented as a working prototype by the end of the term.

4. Software Requirements Specification (SRS)

This section follows the structure of an IEEE-style Software Requirements Specification, covering functional and non-functional requirements and system use cases.

4.1 Functional Requirements

ID	Requirement	Rationale / Notes
FR-01	As a user, I want to authenticate before accessing protected system functions, so that only registered users can access authorized campus-safety features.	Prevents unauthorized access to protected system functions.
FR-02	As an authenticated user, I want system access limited according to my student, safety admin, or floor warden role, so that I can perform only the functions authorized for my role.	Implements the project's three-role role-based access-control model.
FR-03	As a student, I want to submit a hazard report containing a category, building, room/area, and description, so that campus safety staff receive the information required to review the hazard.	Ensures every hazard report contains the required project-scope information.
FR-04	As a student, I want to view the current status of hazard reports I submitted, so that I can track their progress.	Supports student-side report tracking.
FR-05	As a safety admin, I want to view submitted hazard reports in an administrative queue, so that I can identify reports requiring review and action.	Provides the administrative work queue defined by the project scope.
FR-06	As a safety admin, I want to advance a hazard report through submitted, under review, resolved, and closed states, so that report handling follows the defined workflow.	Enforces the approved hazard-report lifecycle.
FR-07	As a safety admin, I want the system to require a resolution note before a hazard report can be marked resolved, so that the reason for resolution is documented.	Enforces the project's mandatory resolution-note rule.
FR-08	As a safety admin, I want to escalate a reviewed severe hazard into an emergency check-in event, so that emergency accountability	Connects hazard review with the emergency-accountability subsystem.

	can begin when a reported condition requires it.	
FR-09	As a student, I want to start a Safe Walk session with a destination and timer when I do not already have an active session, so that my walk can be monitored until completion or expiration.	Implements the one-active-session Safe Walk rule.
FR-10	As a student, I want to extend an active Safe Walk session one time, so that I can add time when my walk takes longer than expected.	Implements the single-extension rule.
FR-11	As a student, I want to mark an active Safe Walk session as arrived or cancelled, so that the system records that monitoring is no longer required.	Provides explicit user-controlled completion or cancellation of a Safe Walk.
FR-12	As a safety admin, I want an expired Safe Walk session to automatically generate a simulated notification and a new incident in the administrative queue, so that an unconfirmed walk receives follow-up.	Implements the required Safe Walk expiry cross-component behavior.
FR-13	As a floor warden, I want to open an emergency check-in event for a specified building, so that students in that building can report their safety status.	Starts the emergency-accountability process for a building.
FR-14	As a student, I want to confirm my safety during an active emergency check-in event, so that I am counted as accounted for.	Provides the student check-in action during an emergency event.
FR-15	As a floor warden, I want to view accounted-for and not-accounted-for student counts for an active emergency event, so that I can monitor emergency accountability.	Provides the live accountability information required by the project objectives.
FR-16	As a floor warden, I want students with active Safe Walk sessions to be flagged in the emergency-accountability view, so that their	Connects Safe Walk information with emergency accountability.

	active walk status is visible during an emergency.	
FR-17	As a student, I want each Safe Walk session to have one defined state—active, arrived, expired, or cancelled—so that I can understand the current status of the session.	Implements the four-state Safe Walk model defined in the project scope.

4.2 Non-Functional Requirements

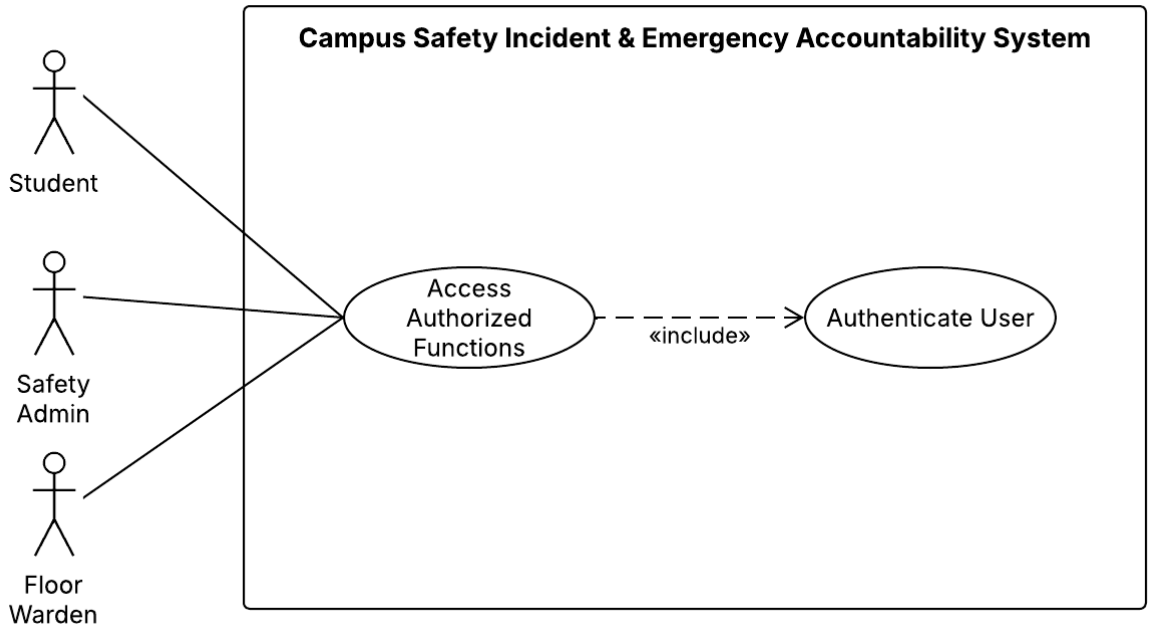
Provide at least one requirement statement for every leaf category of textbook Figure 4.3's non-functional requirements taxonomy (product, organizational, and external requirements). Adjust wording to fit your specific project; not every leaf will be equally significant, but each must be addressed.

NFR Type (Fig. 4.3 leaf)	**Category**	**Requirement Statement**
Performance	Product - Efficiency	The accounted-for/not-accounted-for count shall update within 5 seconds after a student successfully submits an emergency check-in.
Space	Product - Efficiency	The system shall limit each hazard-report description to 2,000 characters and shall reject additional characters beyond this limit.
Reliability	Product - Dependability	The system shall preserve successfully submitted hazard reports, Safe Walk session states, and emergency check-ins so that previously committed data is not lost following an application restart.
Availability	Product - Dependability	The system shall achieve at least 99% availability during published campus operating hours, excluding scheduled maintenance.
Security	Product - Dependability	The system shall require authentication for protected functions and shall restrict actions and data according to the authenticated user's student, safety admin, or floor warden role.
Safety	Product - Dependability	The system shall not dispatch requests to real police, 911, or other emergency services, and all emergency-service responses and alerts generated by the system shall be simulated within the application.
Usability	Product	At least 90% of first-time student users participating in usability testing shall be able to

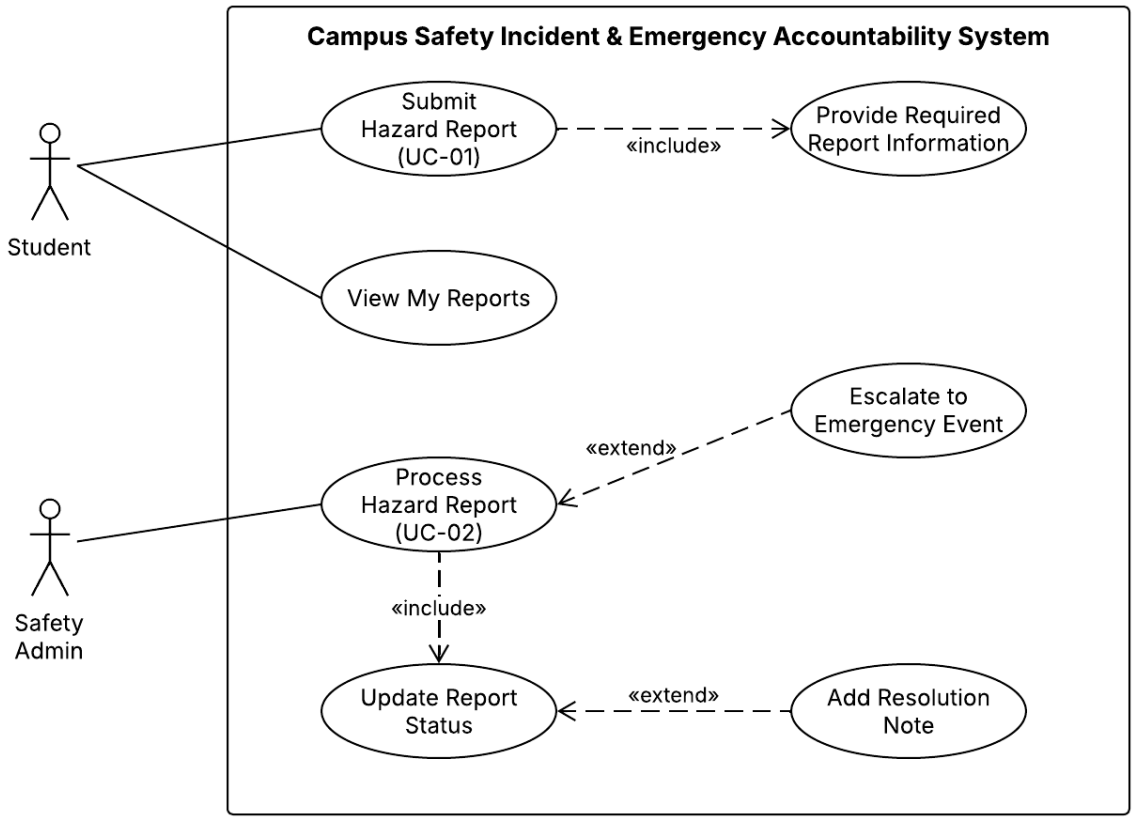
		submit a valid hazard report within 90 seconds without training or assistance.
Environmental	Organizational	The system shall operate through current mobile and desktop web browsers on student-owned devices without requiring dedicated campus hardware.
Operational	Organizational	The system shall enforce the hazard-report workflow submitted → under review → resolved → closed, and shall prevent a report from entering the resolved state unless a resolution note has been recorded.
Development	Organizational	The project shall maintain source code and development artifacts under version control in the team's GitHub repository, and credentials or other authentication secrets shall not be committed to the repository.
Interoperability	External	Data exchanged between system components shall use documented, non-proprietary structured data formats so that the hazard-reporting, Safe Walk, and emergency check-in components can exchange required incident information without dependence on a proprietary client.
Ethical	External	The system shall not perform continuous or background location tracking and shall store only location information explicitly provided by a user, such as a hazard location or stated Safe Walk destination.
Legislative - Privacy	External - Legislative	The system shall not store medical records or HIPAA-applicable documents and shall restrict stored personal information to data required to provide the hazard-reporting, Safe Walk, and emergency check-in functions.
Legislative - Safety	External - Legislative	For each closed hazard report, the system shall preserve the report's creation timestamp, status history, and resolution note so that the handling of the incident can be reviewed.
Legislative - Security	External - Legislative	Access to personally identifying incident information shall be restricted to authorized roles, and the system shall maintain a record of administrative access to such information.

4.3 Use Case Diagram

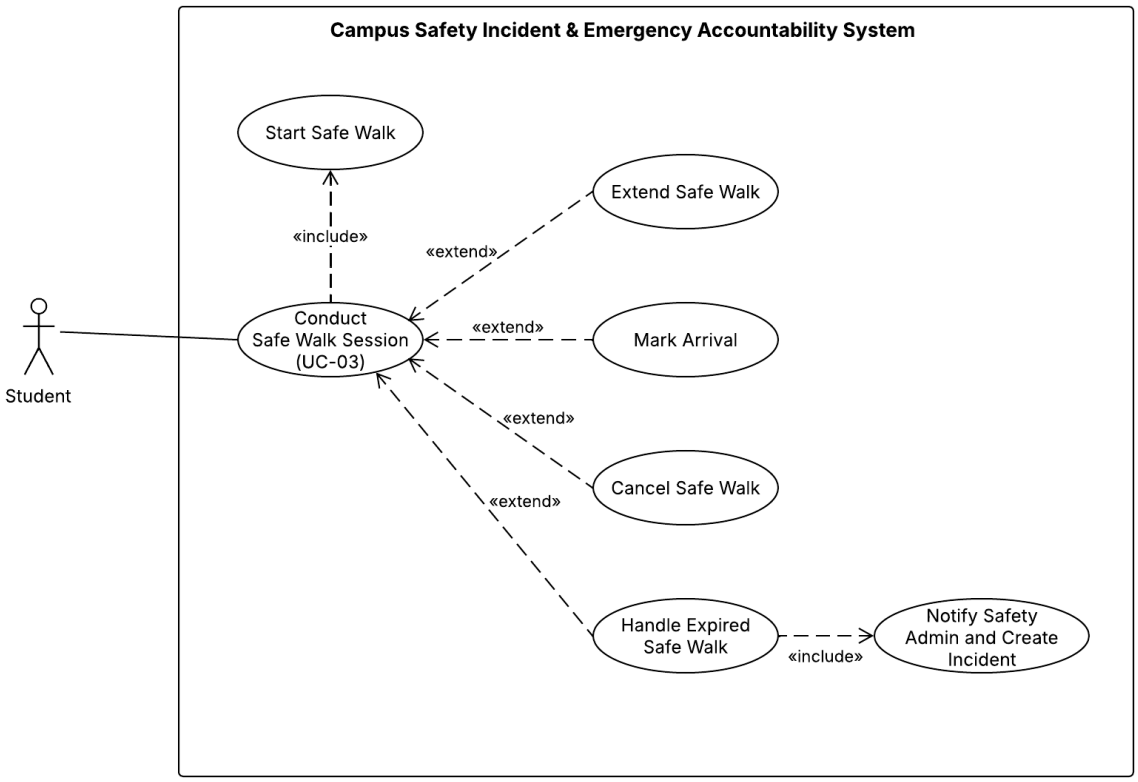
UseCase_Diagram1_Authentication



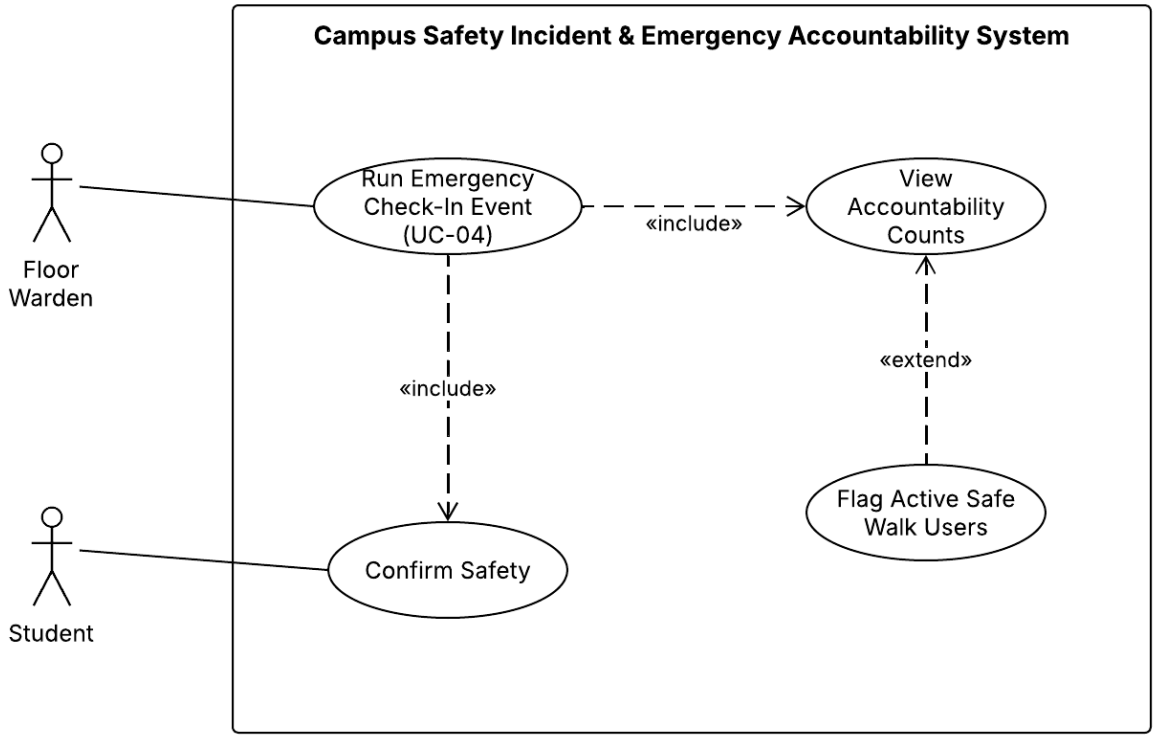
UseCase_Diagram2_Hazard_Reporting


Campus Safety Incident & Emergency Accountability System
Student
Safety Admin
Submit Hazard Report (UC-01)
«include»
Provide Required Report Information
View My Reports
Escalate to Emergency Event
«extend»
Process Hazard Report (UC-02)
«include»
Update Report Status
«extend»
Add Resolution Note

UseCase_Diagram3_Safe_Walk



UseCase_Diagram4_Emergency_CheckIn



Use Case Descriptions

- UC-01 Submit Hazard Report: Actor(s): Student. Goal: Submit a complete campus hazard report and allow the student to track its status. Brief flow: The student enters the hazard category, building, room, or area, and description, then submits the report. The system checks that all required fields are filled in and displays an error if anything is missing. If the report is valid, the system saves it with a Submitted status and shows the student a confirmation and report ID.
- UC-02 Process Hazard Report: Actor(s): Safety Admin. Goal: Review and process submitted hazard reports through the defined administrative workflow. Brief flow: An admin can enter the hazard queue if authorized. An admin can edit the status of a hazard displayed on the admin view. Each change made by the admin is saved in the central database, so anyone accessing the queue will see the changes. If a hazard is marked severe by an admin, the warden is notified, and an emergency event is created automatically.
- UC-03 Conduct Safe Walk Session: Actor(s): Student. Goal: Allow a student to use a monitored Safe Walk session from start until arrival, cancellation, or expiration. Brief flow: A student can create a Safe Walk session, which creates a monitored, timed walk. During the walk, the student can request a timer extension if they need extra time to reach their destination. The session ends when the student marks that they have arrived at their destination or cancels the safe walk. If the timer expires, a notification is sent and an administrative incident report is created.
- UC-04 Run Emergency Check-In Event: Actor(s): Floor Warden, Student. Goal: Open a building emergency event, collect student safety confirmations, and provide current accountability information. Brief flow: A floor warden opens an emergency event for a building, which saves the event and sends a simulated “Are you safe?” prompt to students. When a student confirms, the system rejects the check-in if the event is closed,

ignores it if the student already checked in, and otherwise records a CheckIn and pushes the updated accounted-for and not-accounted-for counts to the warden's board within 5 seconds. The board lists students with an active Safe Walk first, and when the warden closes the event, the remaining not-checked-in list shows who staff need to go look for.

4.4 Requirements Traceability Matrix

The Requirements Traceability Matrix (RTM) links **each functional requirement** back to the use case that generated it, forward to the design element(s) that implement it, and to the method by which it will be verified. It strengthens the report by demonstrating that every requirement is accounted for in the design, and it lays groundwork for the test plan required in Deliverable #2.

Req ID	Requirement (Summary)	Source (Use Case)	Design Element (Class/Component)	Verification Method
FR-01	Authenticate users before protected access	UC-01 Submit Hazard Report	AuthController, User	Test valid and invalid login attempts.
FR-02	Restrict access according to assigned user role	UC-02 Process Hazard Report	AuthController, User, Role, SafetyAdmin, AdminConsole	Attempt authorized and unauthorized role actions and verify access is enforced.
FR-03	Submit hazard with category, building, room/area, and description	UC-01 Submit Hazard Report	Student, StudentApp, ReportController, HazardReport, DataStorage	Submit a complete report, then omit each required field and verify incomplete submissions are rejected.
FR-04	Student views status of own hazard reports	UC-01 Submit Hazard Report	Student, StudentApp, ReportController, HazardReport, DataStorage	Submit a report and verify the submitting student can view its current status.
FR-05	Safety admin views administrative report queue	UC-02 Process Hazard Report	SafetyAdmin, AdminConsole, ReportController, HazardReport, DataStorage	Submit a hazard report and verify it appears in the safety-admin queue.
FR-06	Advance reports through submitted, under review, resolved, and closed states	UC-02 Process Hazard Report	SafetyAdmin, AdminConsole, ReportController, HazardReport, ReportStatus, DataStorage	Exercise valid workflow transitions and verify invalid transitions are rejected.

FR-07	Require a resolution note before resolving a report	UC-02 Process Hazard Report	SafetyAdmin, AdminConsole, ReportController, HazardReport, ReportStatus, DataStorage	Attempt to resolve a report without a note, then with a note; verify only the latter succeeds.
FR-08	Escalate a severe hazard to an emergency check-in event	UC-02 Process Hazard Report	SafetyAdmin, AdminConsole, ReportController, EmergencyController, HazardReport, EmergencyEvent, DataStorage	Escalate a reviewed severe hazard and verify an emergency check-in event is created.
FR-09	Start one active Safe Walk with destination and timer	UC-03 Conduct Safe Walk Session	Student, StudentApp, SafeWalkController, SafeWalkSession, SafeWalkState	Start one Safe Walk, then attempt to start a second while the first remains active; verify the second is rejected.
FR-10	Extend an active Safe Walk one time	UC-03 Conduct Safe Walk Session	Student, StudentApp, SafeWalkController, SafeWalkSession, SafeWalkState, SystemTimer	Extend an active session once and verify a second extension is rejected.
FR-11	Mark an active Safe Walk arrived or cancelled	UC-03 Conduct Safe Walk Session	Student, StudentApp, SafeWalkController, SafeWalkSession, SafeWalkState	Mark separate active sessions arrived and cancelled and verify the resulting states.
FR-12	Expired Safe Walk generates a simulated notification and admin incident	UC-03 Conduct Safe Walk Session	StudentApp, AdminConsole, SystemTimer, SafeWalkController, SafeWalkSession, SafeWalkState, NotificationService, ReportController, HazardReport, DataStorage	Allow an active Safe Walk timer to expire and verify a simulated notification and new admin-queue incident are created.
FR-13	Floor warden opens an emergency event for a specified building	UC-04 Run Emergency Check-In Event	FloorWarden, WardenConsole, EmergencyController, EmergencyEvent, DataStorage	Open an event for a building and verify an active emergency event is created.
FR-14	Student confirms safety during an	UC-04 Run Emergency	Student, StudentApp, EmergencyController,	Submit a student safety confirmation and verify the check-in is

	active emergency event	Check-In Event	EmergencyEvent, CheckIn, DataStorage	recorded as accounted for.
FR-15	Display accounted-for and not-accounted-for counts	UC-04 Run Emergency Check-In Event	FloorWarden, WardenConsole, EmergencyController, EmergencyEvent, CheckIn, DataStorage	Record student check-ins and verify the accountability counts update correctly.
FR-16	Flag students with active Safe Walk sessions during an emergency	UC-04 Run Emergency Check-In Event	FloorWarden, WardenConsole, EmergencyController, SafeWalkController, SafeWalkSession, EmergencyEvent	Create an active Safe Walk, open an emergency event, and verify the student is flagged in the accountability view.
FR-17	Maintain Safe Walk states active, arrived, expired, or cancelled	UC-03 Conduct Safe Walk Session	SafeWalkController, SafeWalkSession, SafeWalkState	Exercise each permitted state outcome and verify only the four defined states are stored.

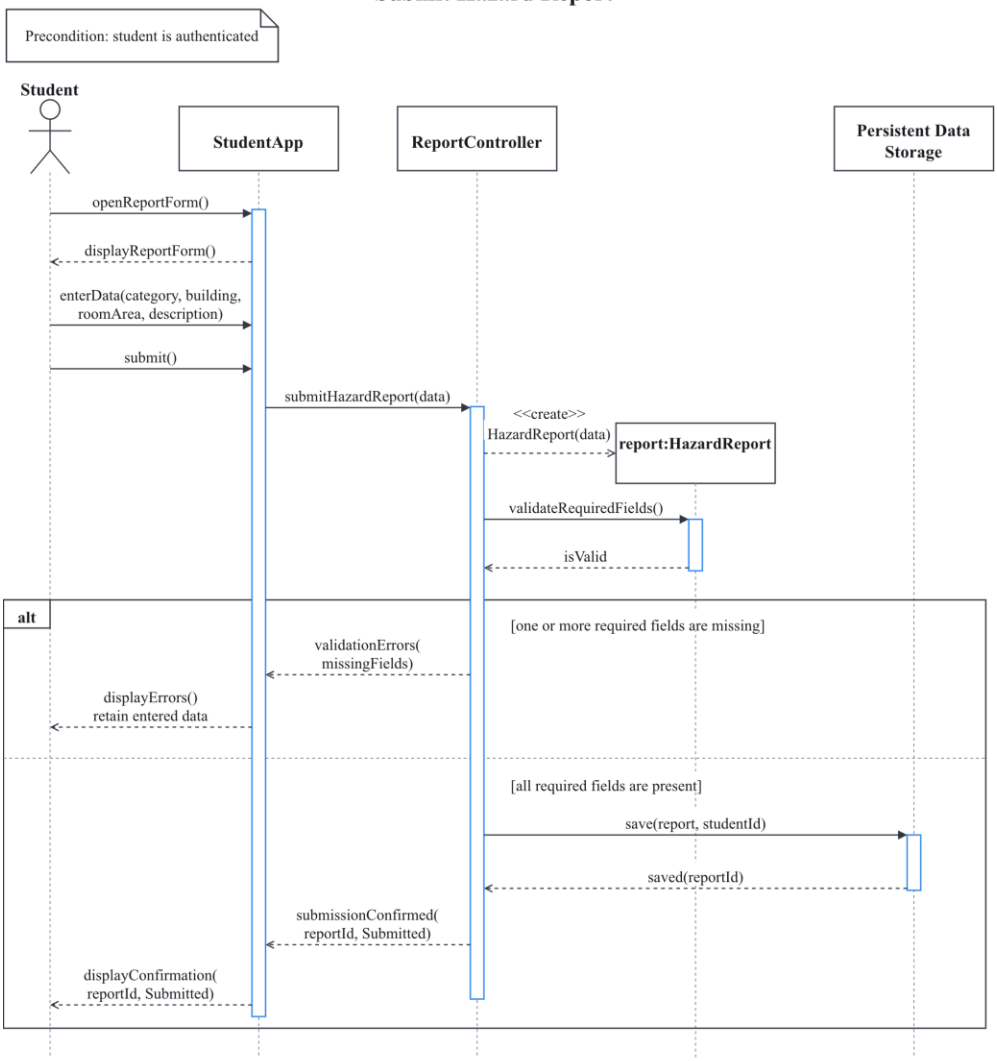
5. Software Design Description (SDD)

This section follows the structure of an IEEE-style Software Design Description, capturing the system's dynamic behavior (sequence diagrams), static structure (class diagram), and architecture.

5.1 Sequence Diagrams

Submit Hazard Report

Submit Hazard Report



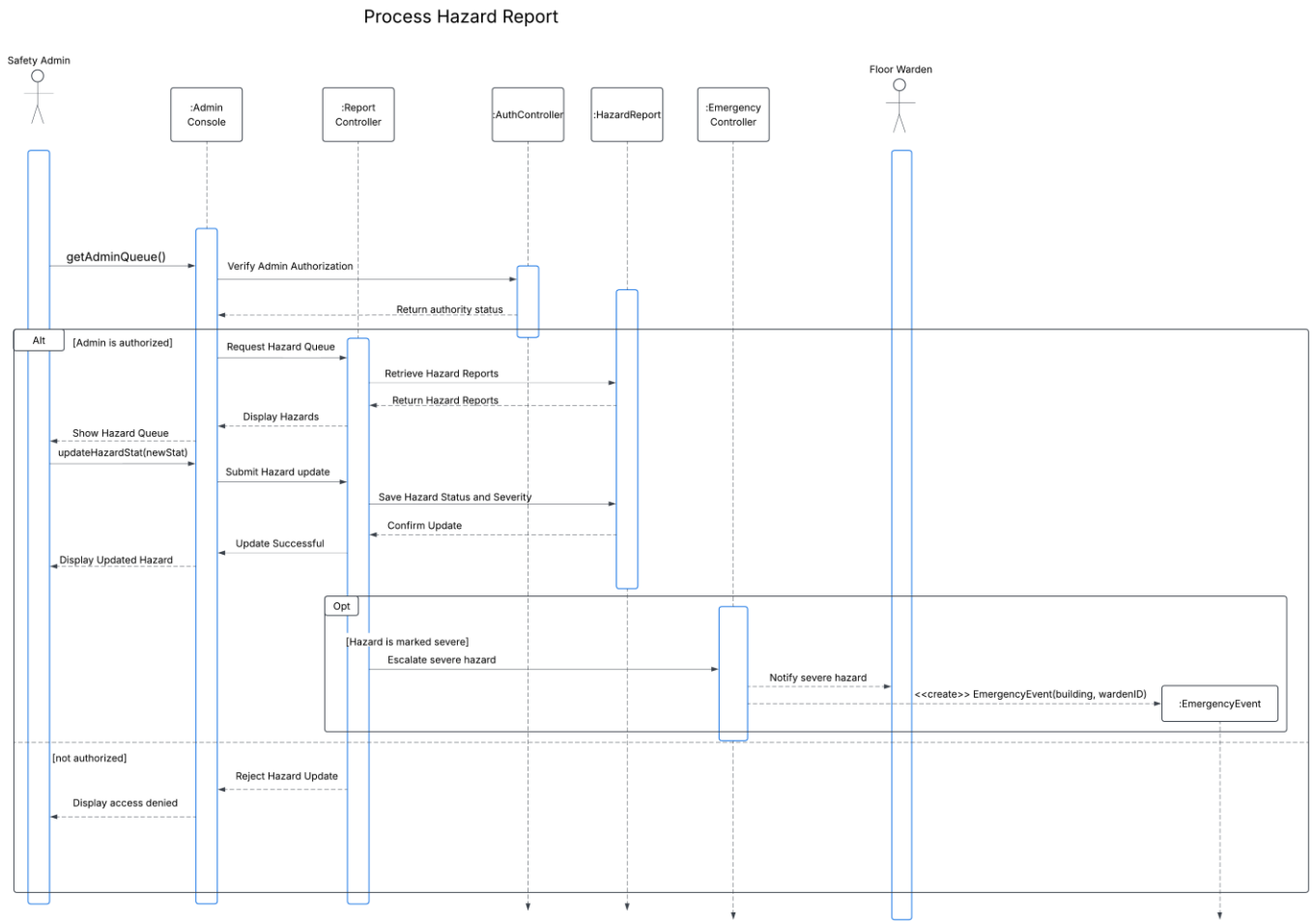
Required data: category, building, room/area, and description.

Success: The saved report is linked to the student. has an ID and creation time.
and starts in Submitted status. It is available in My Reports and the safety admin queue.

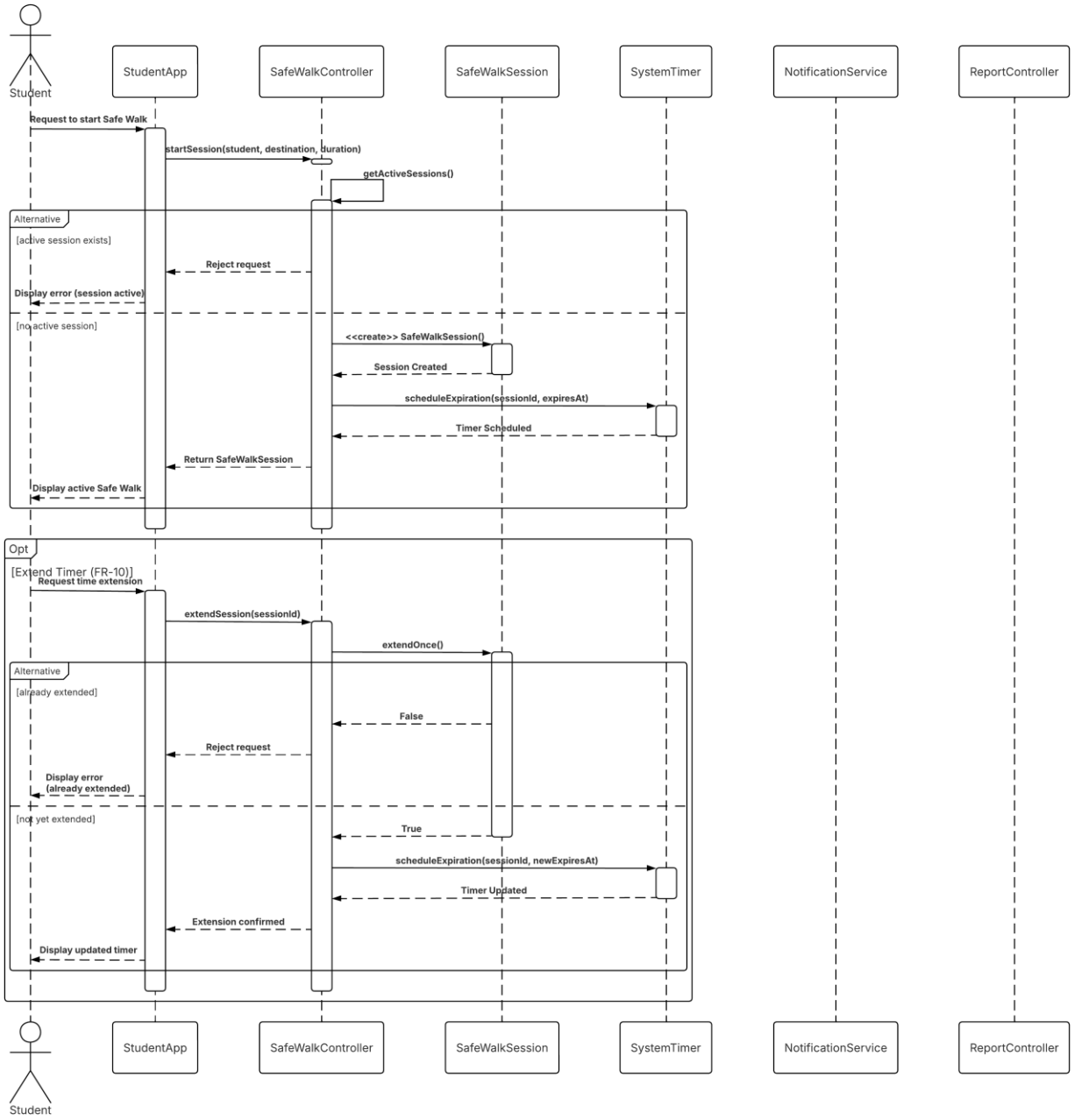
Validation failure: No report is saved; the student can correct the form and resubmit.

Notation: solid arrows = calls; dashed arrows = replies or object creation; blue bars = activations.

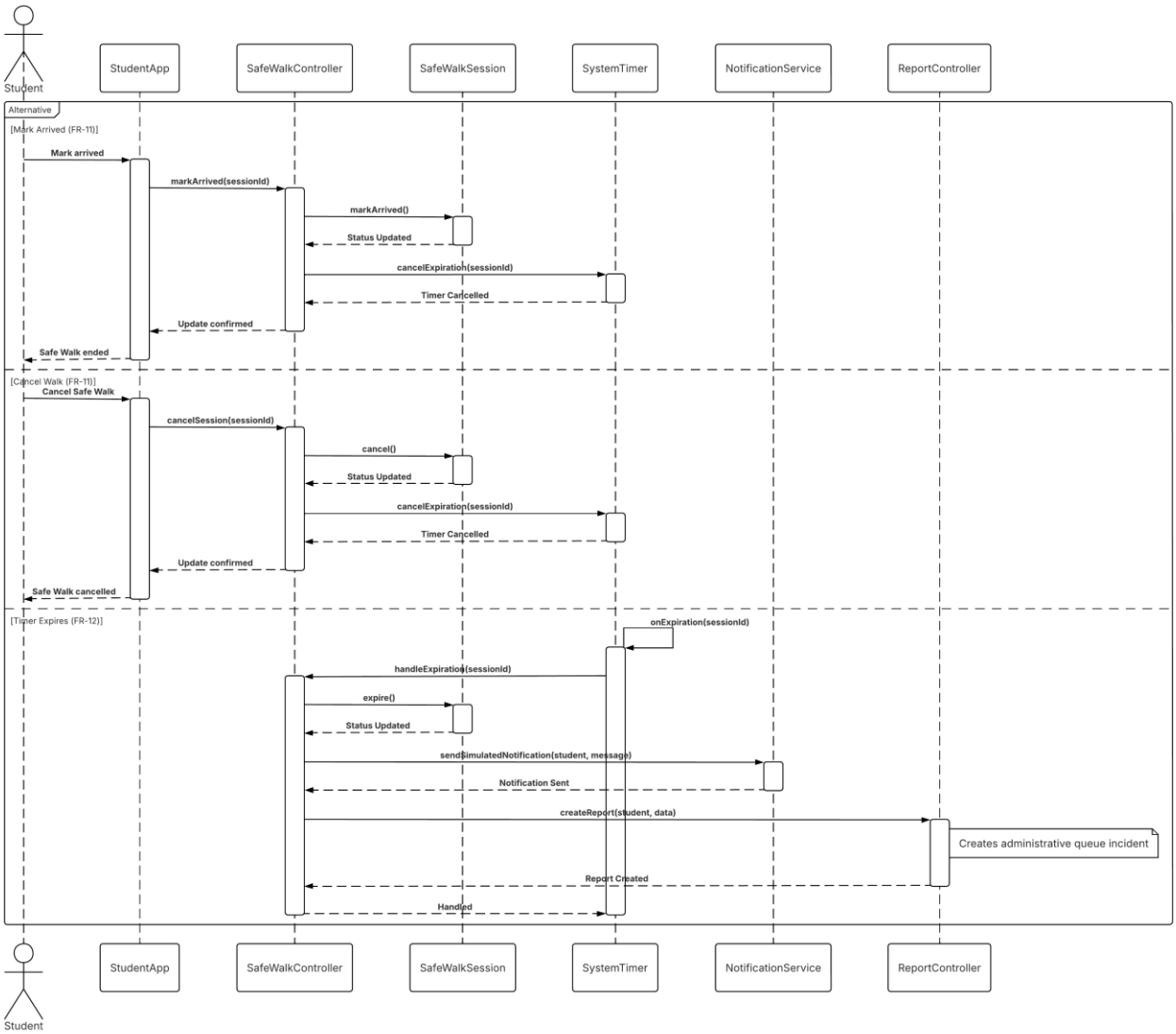
Process Hazard Report



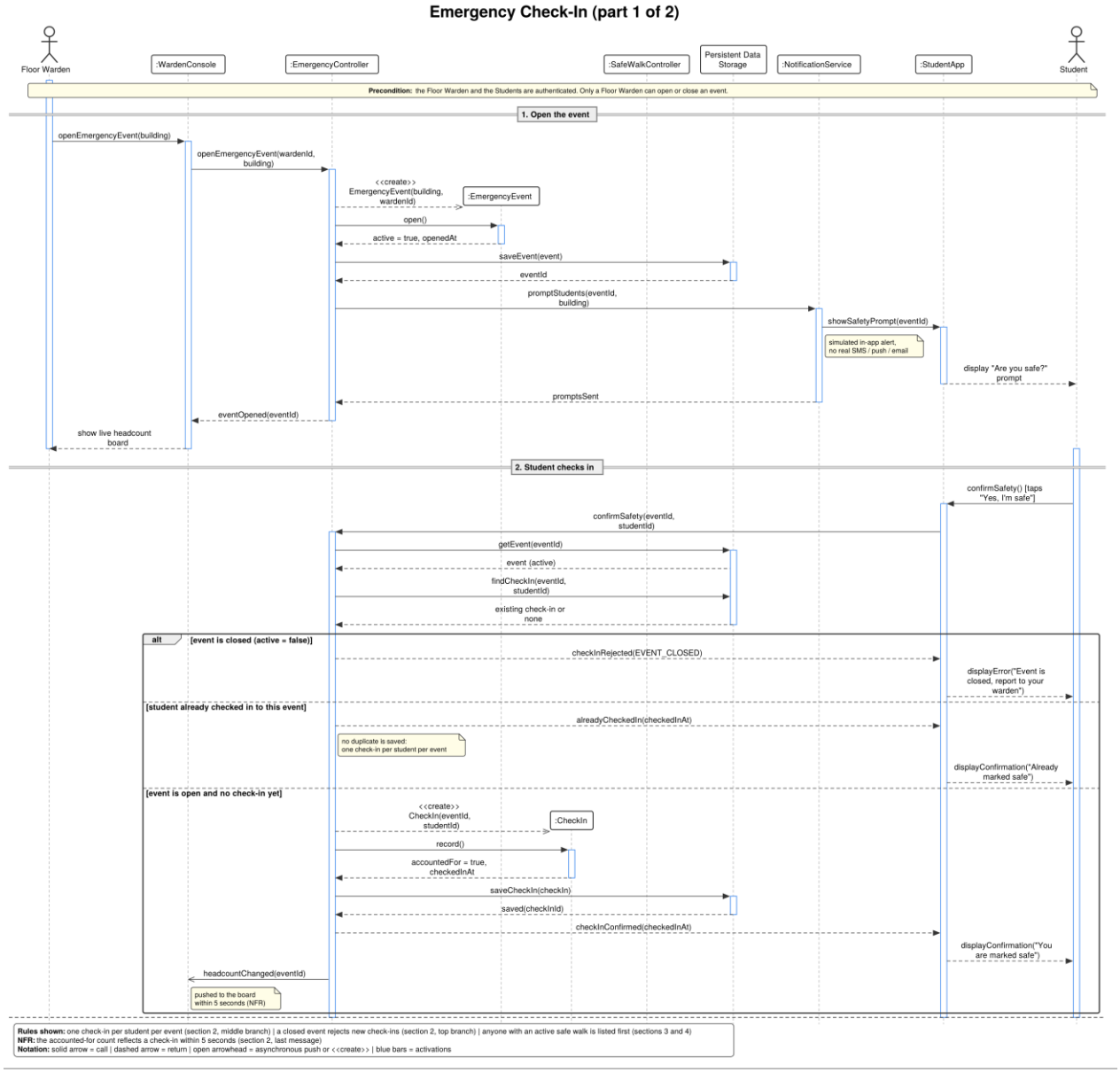
Safe Walk Part 1 of 2



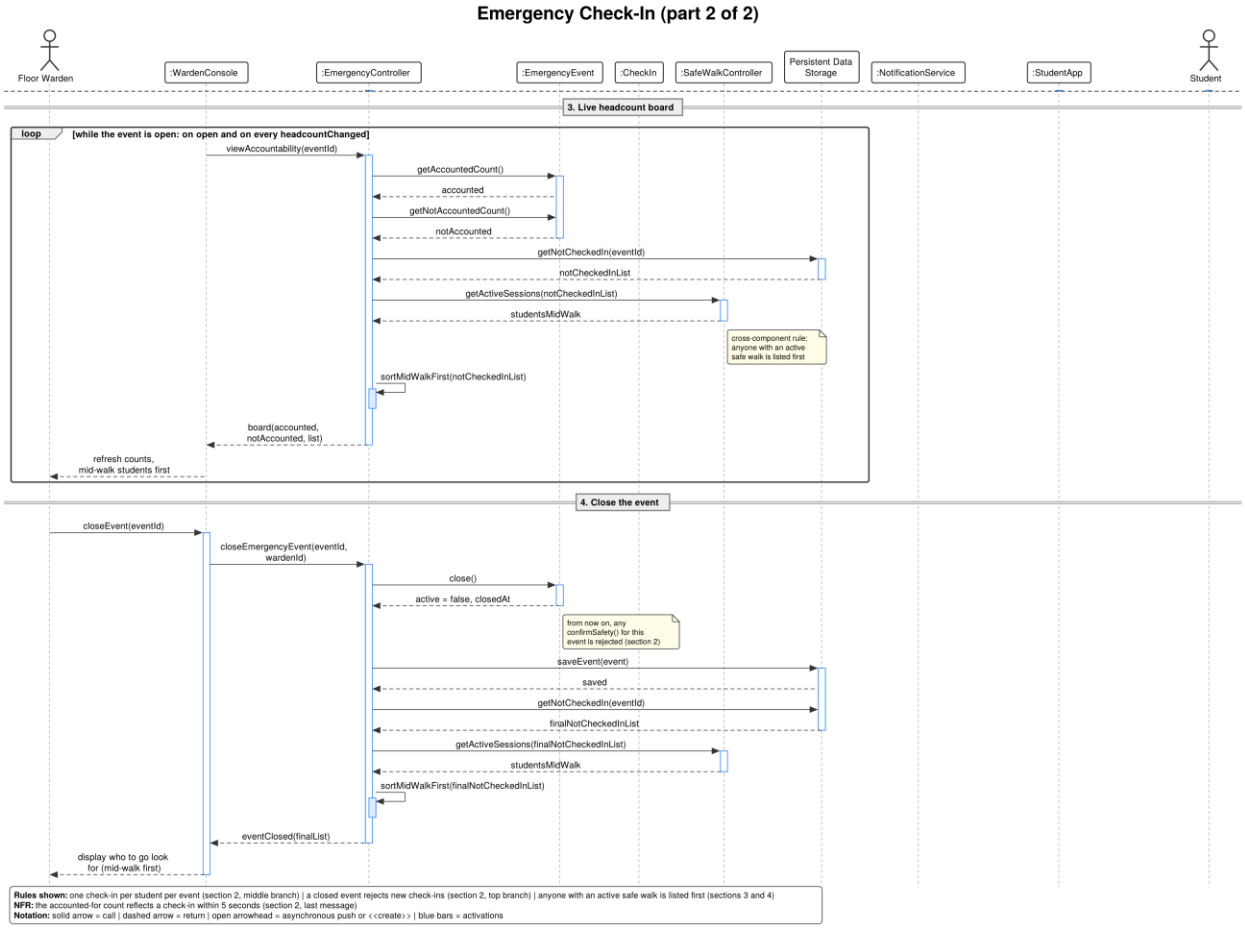
Safe Walk Part 2 of 2



Emergency Check-In Event Report (Part 1)

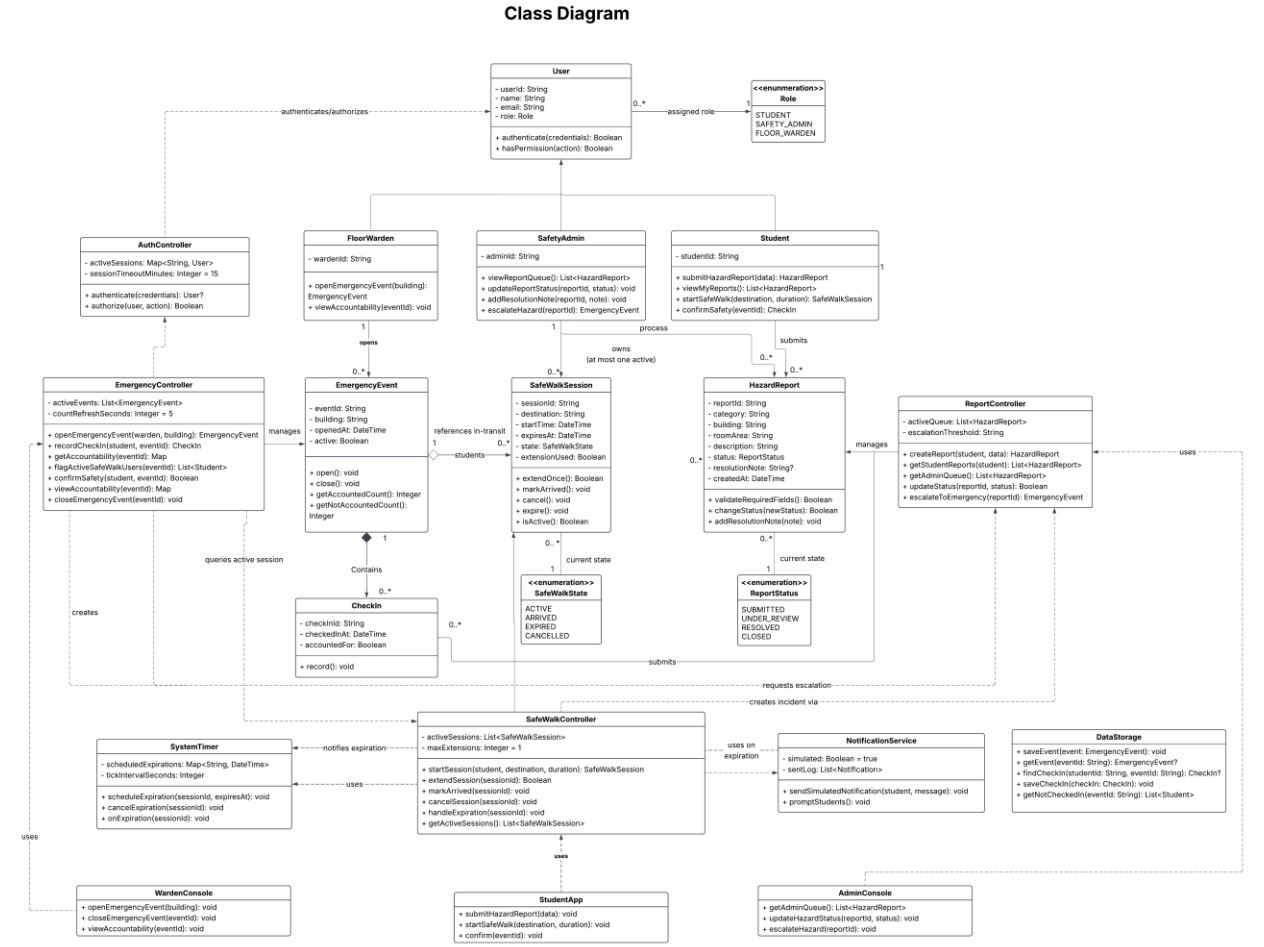


Emergency Check-In Event Report (Part 2)



5.2 Class Diagram

Class Diagram



5.3 Architectural Design

Selected Pattern

Layered architecture.

Diagram

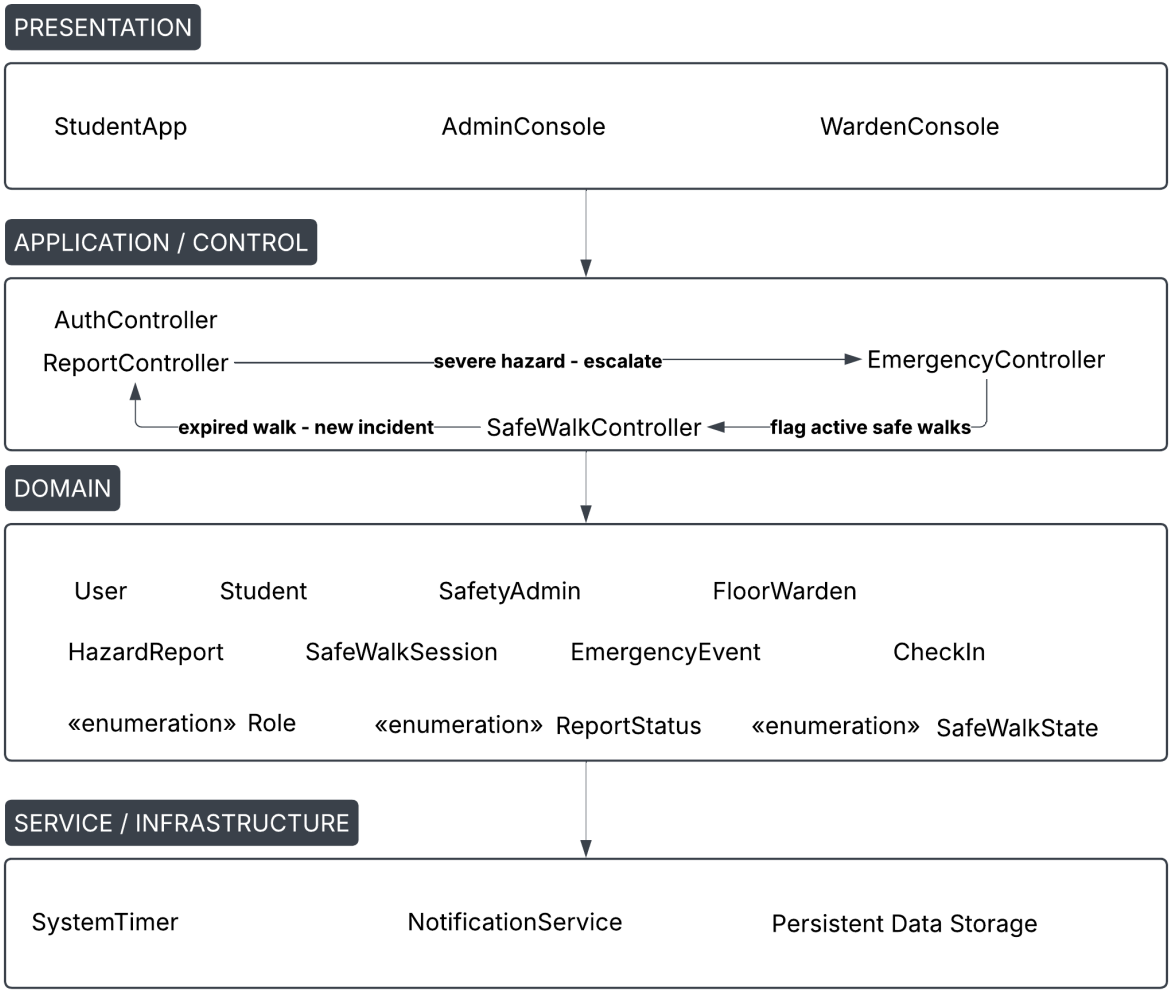


Figure 5.3 Layered architecture of the Campus Safety Incident & Emergency Accountability System. Each layer calls only the layer directly beneath it; the three labeled arrows in the Application/Control layer are the cross-subsystem rules.

Justification

The system is organized into four layers, each depending only on the layer beneath it: Presentation (StudentApp, AdminConsole, WardenConsole), Application/Control (AuthController, ReportController, SafeWalkController, EmergencyController), Domain (the user hierarchy, HazardReport, SafeWalkSession, EmergencyEvent, CheckIn and their enumerations), and Service/Infrastructure (SystemTimer, NotificationService and persistent data storage).

Layered architecture fits because this is one integrated system rather than three separate applications. Hazard reporting, Safe Walk and emergency accountability share one access-control policy and one administrative incident queue, and they interact through three cross-subsystem rules: a severe hazard is escalated to an emergency event, an expired Safe Walk creates a new incident in the administrative queue, and students with an active Safe Walk are flagged during an emergency. All three rules live in the Application/Control layer as controller-to-controller calls, so the coordination logic sits in one place instead of being scattered across user interfaces or data code.

The layering also matches how the team divided the work. Each feature slice owns its own controller, while the domain and infrastructure layers are shared. Authentication and role checks are enforced once in AuthController, so no presentation component can bypass them — this is how the design protects a student's Safe Walk destination, which only the student and authorized safety staff may read.

An expired Safe Walk does not use a separate incident class; SafeWalkController creates an administrative queue item through ReportController, the same kind of item a submitted hazard report produces. The subsystems therefore share a single incident state, which is the integration established in the project charter.

Alternatives considered: Repository architecture was the closest alternative, since the system does rely on shared data, but it describes only how data is shared and not the access-control and coordination logic that carries this project's complexity. Model-View-Controller organizes user-interface code rather than cross-subsystem behavior and would split the design into three separate controller clusters. Client-server describes almost any web deployment, so it would not explain anything specific to this system. Pipe and filter was rejected because the system has no sequential data-transformation pipeline.

---

6. GitHub Repository Setup

Complete the following setup tasks in your team's GitHub repository. These are checked against your commit history, so timing and authorship matter.

Tasks 1.3, 1.4, and 1.5 must each be performed by a different team member - commit history will be checked to confirm this.

Do NOT include credentials (e.g., UTD ID) anywhere in the repository.

Only commits made before the deadline will be considered - push your changes in time.

Task #	Task Description	Done By
1.1	Each team member should create a GitHub account if they do not already have one.	All team members
1.2	Create a GitHub repository named **F26_3354_TeamNumber**.	Kourosh Torkaman Sohrabi
1.3	Add all team members, and the TA/Grader, as collaborators. (TA/Grader GitHub ID will be provided once assigned.)	Kourosh Torkaman Sohrabi
1.4	Make the first commit to the repository: a **README file** with [Team Number and Team Name] as its content.	Neelesh Polireddy (commit 65406da)
1.5	Make another commit including a **pdf** file named "Project Scope." This file’s content should be identical to your topic's	Quan Do (commit 40ef02a)

	description that your team chose from one of the predefined topics. (Section 4 of your Project Charter)	
1.6	Include the **URL** of your team's project repository in your Deliverable #1 report.	Kourosh Torkaman Sohrabi (URL in Section 0)